

# FOSS United Grant Application

A Generalisable Pipeline for Backporting ROCm  
to Obsolete AMD Hardware

**Applicant:** Soham Kulkarni    **Email:** [netherquark@protonmail.com](mailto:netherquark@protonmail.com)

**Substack:** [netherquark.substack.com](https://netherquark.substack.com)    **GitHub:** [github.com/netherquark](https://github.com/netherquark)

**Affiliation:** FOSS Club at Symbiosis Institute of Technology, Pune. 23-27 batch.

## Summary

---

ROCm is the only credible open-source alternative to CUDA for GPU compute. As it matures, AMD periodically drops support for older architectures. For example, GCN5 (Vega) was cut at ROCm 5.7. This leaves a large installed base of capable hardware stranded, with no clear upgrade path and no community resource explaining how to recover one.

This project aims to build a **reusable backporting pipeline**: a systematic, benchmarked process for getting any dropped AMD GPU architecture working with newer ROCm versions. The pipeline covers environment setup, architecture spoofing, userspace/kernel version decoupling, containerised fallbacks, and source builds. Performance benchmarking at every stage will help characterise the real cost of each approach. The final goal is to allow a developer with any other obsolete AMD GPU to apply the same process to their hardware without starting from scratch.

The two initial hardware targets are:

- **gfx90c** — Ryzen 5 5650U (Cezanne APU, Vega 7, 7 CUs, XNACK-enabled, Thinkpad L14 Gen2a, Fedora 43, system ROCm 6.4.4)
- **gfx902** — Ryzen 5 2200G (Raven Ridge, Vega 8, 8 CUs, XNACK-enabled, desktop with 2200G, RHEL 9.5)

These cover the laptop and desktop form factors of GCN5 and represent a wide cross-section of AMD hardware (MI25, Vega 10, Vega 12, Vega 20, Raven Ridge, Picasso, Renoir, Cezanne) in circulation in India.

All pipeline stages, benchmark data, and reproduction steps are published openly on [Substack](https://netherquark.substack.com) under CC-BY-SA. Code will be published on GitHub under GPL-v3 licensing.

## Community Impact

---

GCN5 APUs are common in the Indian student and developer demographic, and platform upgrades have been put on hold due to inflated GPU/RAM prices. Ryzen laptops and desktops are widespread and affordable. The existing community resources for getting ROCm working on this hardware amount to fragmented Reddit threads and closed GitHub issues. No reproducible, end-to-end reference exists for a modern Linux distribution. This project aims to fulfil that gap in a way that outlasts the specific hardware. The pipeline — enumerate the strategies, benchmark each viable path, document the failure modes — applies identically when ROCm eventually drops RDNA1 or any other architecture. The GCN5 case is the first traversal; the methodology is the lasting contribution. The work done will help enable the following hardware:

- APUs with Raven Ridge, Picasso, Renoir, Cezanne architecture. This includes any AMD laptop and desktop with a "Vega" series integrated GPU.
- Radeon MI25/50/60 Instinct server grade GPUs (crucial for scientific computing)
- Vega 10, 12, 20 discrete GPUs (packaged as RX Vega 56/64 or Radeon VII respectively)

## Problem Statement

---

AMD’s ROCm stack is currently the only viable open-source alternative to CUDA for general-purpose GPU computing. However, as ROCm evolves, AMD periodically deprecates support for older GPU architectures. This creates a structural gap: **perfectly functional hardware loses access to modern software ecosystems despite retaining sufficient compute capability.**

This problem is especially acute in contexts such as India, where a large number of students and developers rely on Ryzen APUs and older Vega-based GPUs due to cost constraints. These systems are widely available, energy-efficient, and capable of meaningful ML and compute workloads. Yet, once support is dropped:

- Modern frameworks such as PyTorch fail due to missing precompiled kernels (e.g., absence of `gfx90c` support in rocBLAS).
- Official documentation provides no migration or recovery path.
- Existing community knowledge is fragmented across forum threads, issue trackers, and undocumented workarounds.

While multiple recovery strategies exist — including architecture spoofing, mixed-version stacks, containerisation, and source builds — they are **not systematised, not benchmarked, and not reproducible**. Users attempting to revive unsupported hardware must rediscover the same failure modes independently, often without understanding whether limitations arise from drivers, userspace libraries, or application-level constraints.

The absence of a structured approach leads to three core issues:

1. **Knowledge fragmentation:** No single resource maps the full space of recovery strategies or explains their trade-offs.
2. **Lack of reproducibility:** Even successful configurations are difficult to replicate across systems or GPU variants.
3. **Unclear feasibility boundaries:** It is not known which approaches can reliably restore functionality, what performance penalties they incur, or where recovery fundamentally breaks down.

As a result, a large installed base of AMD hardware is effectively excluded from modern open-source compute workflows — not due to hardware limitations, but due to missing integration pathways.

This project addresses the need for a **systematic, generalisable framework** that transforms ad-hoc recovery attempts into a reproducible pipeline. By enumerating all viable strategies, experimentally validating them, and documenting both successes and failure modes, the work aims to convert a fragmented problem space into a structured, reusable methodology.

## Proposed Solution

---

When AMD drops a GPU architecture from ROCm, the available recovery strategies are not arbitrary. They fall into a well-defined set that applies to *any* dropped target:

1. **Community recovery paths (baseline workarounds):** Begin with existing community-discovered methods such as `HSA_OVERRIDE_GFX_VERSION` and environment-level tweaks. These approaches require no system modification and establish how far unsupported hardware can be pushed using prebuilt binaries alone.
2. **Userspace/kernel decoupling (mixed-version stacks):** Combine ROCm userspace from one version with a different AMDGPU kernel driver. This isolates whether failures arise from driver incompatibility, missing architecture support in userspace libraries, or ABI mismatches.
3. **Matched system stack validation:** Run a fully matched ROCm + AMDGPU stack for newer versions to confirm that failures are due to dropped architecture support rather than misconfiguration. This stage also includes injecting missing kernels into prebuilt libraries to test partial recovery.
4. **Containerised vendor stacks:** Use AMD-provided Docker images across multiple ROCm versions (5.7, 6.4.4, 7.1). This determines whether containerisation can preserve support for dropped hardware or expose functionality unavailable in the host system.
5. **Application-level source builds:** Rebuild PyTorch against specific ROCm versions (5.7, 6.4.4, 7.1) with explicit `gfx*` targets. This tests whether application-layer recompilation alone can recover functionality without modifying the full ROCm stack.
6. **Full system stack source builds:** Recompile ROCm components (HIP runtime, rocBLAS, MIOpen, etc.) with explicit architecture enablement. This stage attempts native kernel generation and represents the most complete recovery strategy.
7. **Cross-architecture validation:** Repeat all successful configurations on a second target (`gfx902`) to determine whether the pipeline generalises across multiple GPUs within the same architectural family.
8. **Forward-porting attempts (exploratory):** Attempt to extend newer ROCm/AMDGPU versions (e.g., 7.1) to unsupported architectures. This stage is explicitly experimental and intended to probe the limits of feasibility rather than guarantee success.

The pipeline is a **decision tree across these strategies**.

## Benchmarking

---

The `gfx900` spoof presents `gfx90c` as a Radeon Instinct MI25. The MI25 has 64 CUs and HBM2; the Vega 7 has 7 CUs and shared DDR4. Initial results show achieved scores are in the range of 50% of AMD's published figures for the Vega 7. These numbers are not documented anywhere. The same is true for other incomplete/hacky approaches to tackle this problem. In order to find functional solutions which fully utilise the hardware, every viable configuration will be benchmarked against results on the latest stable ROCm release for the target and AMD's published theoretical figures. Both HIP and PyTorch benchmarking will be done for every configuration.

## Prior Work

---

### Post #1 — Context and Motivation

*AMD vis-à-vis GPUs #1: So.. what are they doing about Nvidia's moat?*

Introduces the hardware target, the ROCm ecosystem, and the motivation for running modern ML tooling on dropped GCN5 hardware. Discusses AMD's open-source trajectory and the gap left by support cutoffs.

### Post #2 — Installation and Problem Identification

*AMD vis-à-vis GPUs #2: Installing ROCm & PyTorch on Fedora 43 natively*

Documents native ROCm 6.4.4 installation via Fedora's DNF metapackage and successful HIP program execution, confirming the hardware compute stack is functional. Identifies the core problem: PyTorch's bundled rocBLAS ships no gfx90c kernels. Maps the five pipeline strategies as the paths forward.

## Next steps

---

### Phase #3 — Community Recovery Paths: Spoofing and Baseline Workarounds

This phase will:

- Test community spoofing via HSA\_OVERRIDE\_GFX\_VERSION on gfx90c.
- Check whether spoofing enables direct HIP execution and whether kernels dispatch correctly.
- Run the same spoofed configuration under PyTorch and identify which libraries fail first.
- Establish a baseline for how far unsupported hardware can be pushed without rebuilding or downgrading the system.
- Research and test other recovery strategies.

### Phase #4 — ROCm 5.7 Userspace on ROCm 6.4.4 AMDGPU

This phase will:

- Install ROCm 5.7 userspace on a system using the 6.4.4 AMDGPU stack.
- Test direct HIP programs to determine which parts of the older userspace remain functional.
- Run PyTorch against the same mixed stack and identify whether it fails differently from HIP.
- Separate failures caused by userspace compatibility from failures caused by GPU architecture support.
- Run ROCm 6.4.4 userspace on the 6.4.4 AMDGPU system stack.
- Verify direct HIP functionality on gfx90c.
- Run PyTorch and identify whether the workload fails because ROCm 6.4.4 does not ship gfx90c kernels.
- Compile gfx90c kernels and place them in prebuilt ROCm / PyTorch, then test compatibility and performance.
- Use this as the matched-stack comparison point for later container and source-build tests.

#### **Phase #5 — ROCm 5.7, 6.4.4, and 7.1 Docker on ROCm 6.4.4 AMDGPU**

This phase will:

- Run AMD's ROCm 5.7, 6.4.4, and 7.1 Docker images on the 6.4.4 AMDGPU host.
- Test direct HIP workloads inside each container and compare them with the corresponding host or system installs.
- Run PyTorch inside each container and compare behaviour across the three ROCm versions.
- Document host-device exposure, container permissions, and library/runtime interactions.
- Measure whether containerisation preserves the last supported stack cleanly enough to be practical, and whether later versions recover `gfx90c` support.

#### **Phase #6 — gfx902 Repeat: Community and System-Side Recovery Paths**

This post will:

- Re-run spoofing via `HSA_OVERRIDE_GFX_VERSION` on `gfx902`.
- Test direct HIP execution and note whether behaviour differs from `gfx90c`.
- Run PyTorch under the same spoofed and mixed-version conditions.
- Compare the failure modes of `gfx902` with `gfx90c`.

#### **Phase #7 — gfx902 Repeat: Containerised Recovery Paths**

This phase will:

- Run ROCm 5.7, 6.4.4, and 7.1 Docker images on the same host configuration used earlier.
- Test both direct HIP and PyTorch inside each container.
- Compare container behaviour on `gfx902` with the `gfx90c` results.
- Record whether any container version exposes support that the native stack does not.

#### **Phase #8 — ROCm 5.7 PyTorch from Source on ROCm 6.4.4 AMDGPU**

This phase will:

- Compile PyTorch against ROCm 5.7 with explicit architecture targeting.
- Run the resulting PyTorch build on `gfx90c` and identify whether it is usable.
- Compare the source build against the prebuilt 5.7 wheel and the containerised 5.7 setup.

#### **Phase #9 — ROCm 6.4.4 PyTorch from Source on ROCm 6.4.4 AMDGPU**

This phase will:

- Compile PyTorch against ROCm 6.4.4 with explicit architecture targeting.
- Test direct HIP dependencies exposed through the build and runtime environment.
- Run the resulting PyTorch build on `gfx90c` and identify whether it can produce a working binary.
- Compare the source build against the prebuilt 6.4.4 wheel and the containerised 6.4.4 setup.

#### **Phase #10 — ROCm 6.4.4 System Stack from Source on ROCm 6.4.4 AMDGPU**

Rebuild the ROCm 6.4.4 system stack from source and test it on the 6.4.4 AMDGPU host. Low chance of success - vendor level effort.

This phase will:

- Compile ROCm 6.4.4 components from source with the relevant target architecture enabled.
- Add support for pieces missing due to packaging.
- Test direct HIP execution on `gfx90c`.
- Run PyTorch against the rebuilt stack and compare the result with the source-built PyTorch.

#### **Phase #11 — gfx902 Validation for Source-Built Stacks**

Take the source-built ROCm and PyTorch configurations that work on `gfx90c` and test them on `gfx902`.

This phase will:

- Re-run the successful source-built ROCm 5.7 and ROCm 6.4.4 stacks on `gfx902`.
- Test direct HIP programs under each stack.
- Run PyTorch under each stack and compare behaviour with `gfx90c`.
- Record which source-built configurations generalise cleanly across both chips.

#### **Phase #12 — ROCm 7.1 PyTorch from Source on ROCm 6.4.4 AMDGPU**

Build PyTorch from source against ROCm 7.1 and test the resulting binary on the 6.4.4 AMDGPU host. Low chance of success - vendor level effort. Expect silent regressions.

This phase will:

- Compile PyTorch against ROCm 7.1 with explicit architecture targeting.
- Test direct HIP dependencies exposed through the build and runtime environment.
- Run the resulting PyTorch build on `gfx90c` and identify whether it can produce a working binary.
- Compare the source build against the containerised 7.1 setup.

#### **Phase #13 — ROCm 7.1 System Stack from Source on ROCm 6.4.4 AMDGPU**

Attempt to rebuild the ROCm 7.1 system stack from source and test it on the 6.4.4 AMDGPU host. Low chance of success - vendor level effort. Expect silent regressions.

This phase will:

- Compile ROCm 7.1 components from source with the relevant target architecture enabled.
- Test direct HIP execution on `gfx90c`.
- Run PyTorch against the rebuilt stack and compare the result with the source-built PyTorch.
- Record whether the newer stack changes the support boundary or only changes the failure mode.

### Phase #14 — gfx902 Validation for ROCm 7.1 Stacks

Take the source-built ROCm and PyTorch configurations for ROCm 7.1 and test them on gfx902. Low chance of success - vendor level effort. Expect silent regressions.

This phase will:

- Test direct HIP programs under each stack.
- Run PyTorch under each stack and compare behaviour with gfx90c.
- Record which source-built configurations generalise cleanly across both chips.

### Phase #15 — Attempt to port 7.1 AMDGPU to gfx90c/gfx902

Extremely aspirational. Lack the necessary skills to guarantee any success.

## Future Scope: Automation and Upstream Contributions

---

Once the backporting pipeline is stabilised and validated across both gfx90c and gfx902, the next step is to **automate the pipeline and operationalise it for community use**.

This involves building cloud-based workflows (e.g., CI/CD pipelines) that:

- Automatically build ROCm and PyTorch binaries targeting unsupported architectures (e.g., gfx90c, gfx902).
- Package and distribute these binaries in a reproducible and versioned manner.
- Track compatibility across ROCm releases and surface breakages as architectures are deprecated upstream.
- Provide a public artefact repository — effectively acting as a **community-maintained compatibility layer** for dropped AMD hardware.

This transforms the project from documentation into **infrastructure**: instead of each user rebuilding ROCm independently, they can directly consume prebuilt, validated binaries. We can utilise Frappe’s newly launched cloud platform for this usecase.

In parallel, the grant enables a transition from reverse-engineering unsupported stacks to **contributing upstream to ROCm on supported hardware**.

A modern RDNA2 GPU (e.g., RX 6600 / RX 6650 XT) will allow:

- Active testing and validation of ROCm changes on officially supported architectures.
- Contributions to ROCm components such as the HIP runtime, rocBLAS kernel generation, and build/packaging systems.
- Development of tooling and automation that can be upstreamed to improve maintainability and extensibility of ROCm.

The backporting work builds a deep understanding of ROCm’s compiler and runtime stack; the new hardware enables applying that understanding in a way that directly benefits the upstream ecosystem.

Together, these two directions — **automated backporting infrastructure** and **upstream ROCm contributions** — ensure that the project has both immediate community impact and long-term sustainability.

## Grant Amount Requested

---

INR [X]

The grant will be allocated across three primary components:

- **Hardware (RDNA2 GPU):** Purchase of a modern AMD GPU (e.g., RX 6600 / RX 6650 XT) to enable upstream ROCm development and testing on supported architectures. This is essential for contributing patches, validating behaviour, and engaging with the ROCm ecosystem beyond unsupported hardware.
- **Cloud compute and storage:** Funding for cloud infrastructure required to run automated build pipelines for ROCm and PyTorch. This includes CI/CD runners, storage for build artefacts, and bandwidth for distributing binaries. These resources are necessary to convert the pipeline into a usable public service.
- **Stipend:** Support for sustained development effort across the full duration of the project, including experimentation, benchmarking, documentation, and maintenance of automated workflows.

This allocation ensures that the project delivers:

- A reproducible and generalisable backporting pipeline,
- Publicly accessible prebuilt binaries for unsupported hardware,
- And meaningful upstream contributions to ROCm.